# RAPPORT D'ANALYSE DE DONNÉES

*Big Data avec Apache Spark & Hadoop sur Docker*

*-yasmine frikha-*

## TABLE DES MATIÈRES

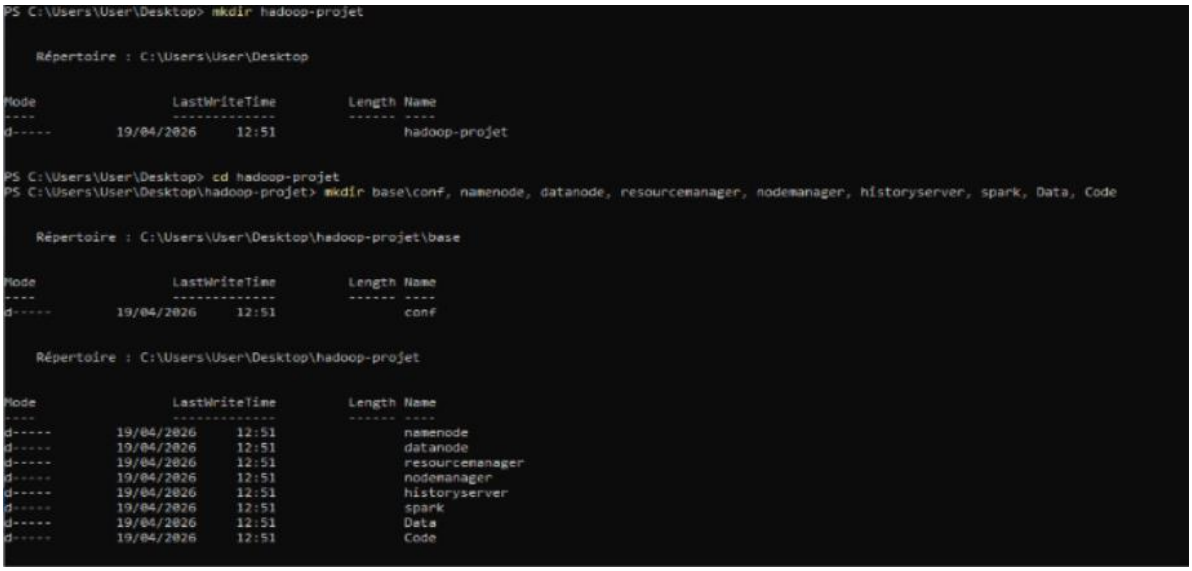
1. Installation du Cluster Hadoop sur Docker
    a. 1.1 Création de la structure du projet
    b. 1.2 Fichiers de configuration XML
    c. 1.3 Fichier hadoop.env
    d. 1.4 Dockerfile de l'image de base
    e. 1.5 Construction des services du cluster
    f. 1.6 Lancement avec docker-compose
2. Application pratique — TP MapReduce
3. Présentation du Dataset COVID-19
4. Définition des KPIs
5. Chargement du Dataset dans HDFS et Exécution Spark
6. Analyse Descriptive avec PySpark
    a. 6.1 Statistiques descriptives globales
    b. 6.2 KPI 1 — Taux de mortalité
    c. 6.3 KPI 2 — Taux de guérison
7. Analyse Prescriptive
    a. 7.1 Résultats de classification
    b. 7.2 Analyse par catégorie
8. Création du Dashboard de Visualisation
    a. 8.1 Installation des bibliothèques
    b. 8.2 Fichiers Python
    c. 8.3 Dashboard résultant
    d. 8.4 Résultats et analyse
9. Limites et Biais du Dataset
10. Conclusion Générale et Perspectives
11. Bibliographie et Références

# 1. Installation du Cluster Hadoop sur Docker

Le cluster Big Data a été déployé localement à l'aide de Docker. Chaque composant Hadoop et Spark tourne dans un conteneur indépendant, simulant fidèlement un environnement distribué réel. Cette approche permet de travailler en local tout en reproduisant la logique d'un vrai cluster de production.

## 1.1 Étape 1 — Création de la structure du projet

On crée le dossier principal hadoop-projet et tous les sous-dossiers nécessaires pour chaque service du cluster (namenode, datanode, resourcemanager, etc.) :




> *Chaque sous-dossier correspond à un service Docker indépendant. Cette séparation permet une gestion modulaire et claire de la configuration.*

## 1.2 Étape 2 — Fichiers de configuration XML

Hadoop utilise 4 fichiers XML de configuration partagés par tous les services du cluster via l'image Docker de base. Ces fichiers définissent les paramètres réseau, de stockage et de gestion des ressources :

| Fichier | Rôle |
|---|---|
| **core-site.xml** | • Adresse du NameNode (hdfs://namenode:9000),<br>• compression SnappyCodec |
| **hdfs-site.xml** | • Facteur de réplication = 3,<br>• chemins stockage NameNode et DataNodes |
| **mapred-site.xml** | • Framework MapReduce = YARN,<br>• mémoire JVM Map (4 Go) et Reduce (8 Go) |
| **yarn-site.xml** | • Chef d'orchestre YARN,<br>• allocation CPU/RAM aux jobs MapReduce et Spark |

Les fichiers sont créés puis configurés dans chaque répertoire de service :

```
PS C:\Users\User\Desktop\hadoop-projet> notepad C:\Users\User\Desktop\hadoop-projet\base\conf\core-site.xml
PS C:\Users\User\Desktop\hadoop-projet> notepad C:\Users\User\Desktop\hadoop-projet\base\conf\hdfs-site.xml
PS C:\Users\User\Desktop\hadoop-projet> notepad C:\Users\User\Desktop\hadoop-projet\base\conf\mapred-site.xml
PS C:\Users\User\Desktop\hadoop-projet> notepad C:\Users\User\Desktop\hadoop-projet\base\conf\yarn-site.xml
PS C:\Users\User\Desktop\hadoop-projet> notepad C:\Users\User\Desktop\hadoop-projet\hadoop.env
PS C:\Users\User\Desktop\hadoop-projet> notepad C:\Users\User\Desktop\hadoop-projet\base\entrypoint.sh
```

core-site.xml

Fichier Modifier Affichage

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://namenode:9000</value>
  </property>
  <property>
    <name>fs.default.name</name>
    <value>hdfs://namenode:9000</value>
  </property>
  <property>
    <name>hadoop.http.staticuser.user</name>
    <value>root</value>
  </property>
  <property>
    <name>io.compression.codecs</name>
    <value>org.apache.hadoop.io.compress.SnappyCodec</value>
  </property>
  <property>
    <name>dfs.client.use.datanode.hostname</name>
    <value>true</value>
  </property>
  <property>
    <name>dfs.datanode.use.datanode.hostname</name>
    <value>true</value>
  </property>
</configuration>
```

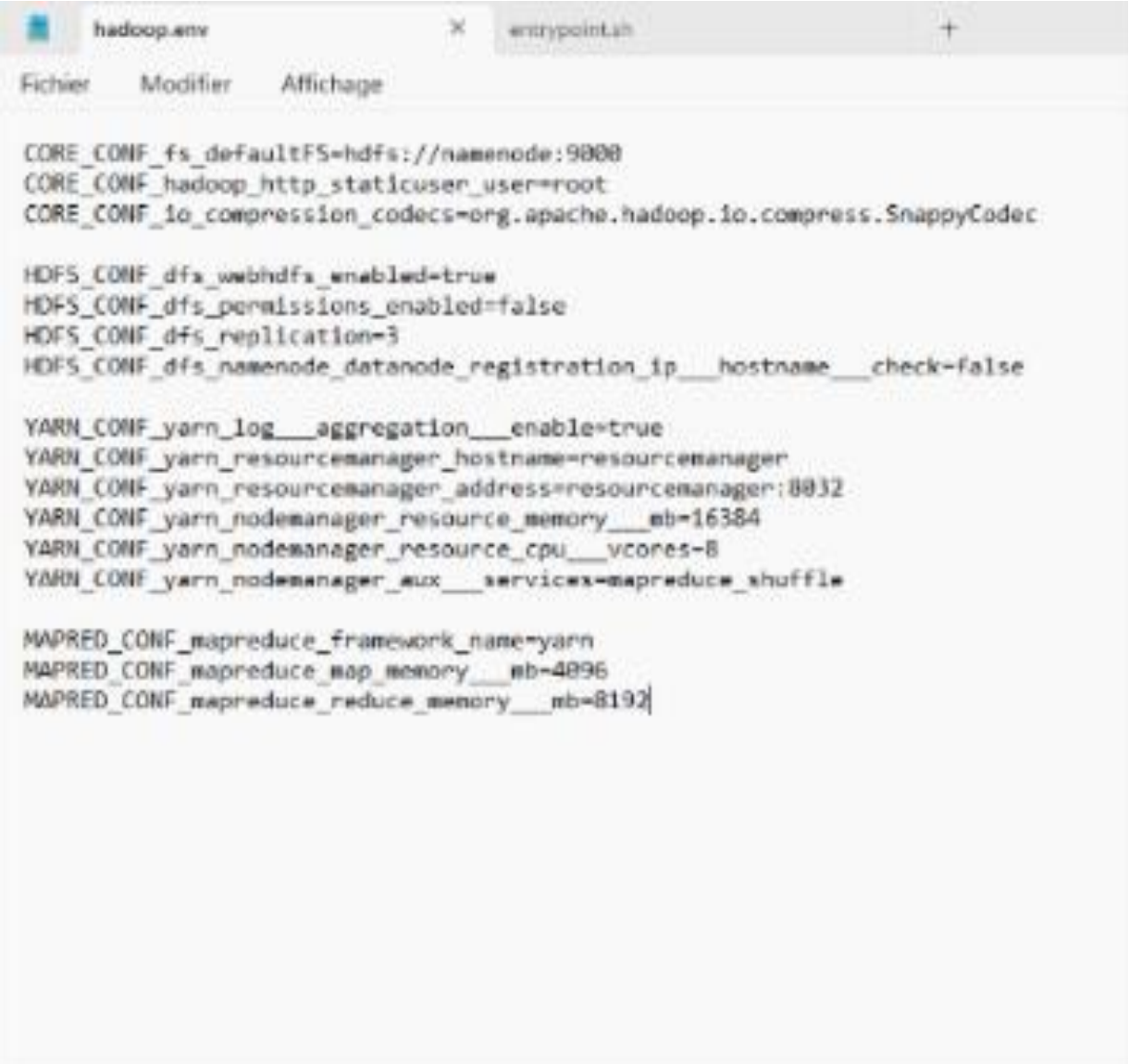
hdfs-site.xml

Fichier Modifier Affichage

```
<configuration>
  <property>
    <name>dfs.webhdfs.enabled</name>
    <value>true</value>
  </property>
  <property>
    <name>dfs.namenode.safemode.min.datanodes</name>
    <value>2</value>
  </property>
  <property>
    <name>dfs.replication</name>
    <value>3</value>
  </property>
  <property>
    <name>dfs.permissions.enabled</name>
    <value>false</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>/hadoop/dfs/name</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>/hadoop/dfs/data</value>
  </property>
  <property>
    <name>dfs.namenode.datanode.registration.ip-hostname-check</name>
    <value>false</value>
  </property>
  <property>
    <name>dfs.namenode.rpc-bind-host</name>
    <value>0.0.0.0</value>
  </property>
  <property>
    <name>dfs.namenode.http-bind-host</name>
    <value>0.0.0.0</value>
  </property>
  <property>
    <name>dfs.client.use.datanode.hostname</name>
    <value>true</value>
  </property>
  <property>
    <name>dfs.datanode.use.datanode.hostname</name>
    <value>true</value>
  </property>
</configuration>
```

## 1.3 Étape 3 — Fichier hadoop.env

Le fichier hadoop.env centralise toutes les variables d'environnement du cluster. Il est chargé par tous les conteneurs au démarrage pour éviter de répéter les configurations dans chaque Dockerfile. Cela garantit la cohérence des paramètres sur l'ensemble du cluster.




## 1.4 Étape 4 — Dockerfile de l'image de base

L'image de base est le fondement de tous les services du cluster. Elle est construite à partir de Debian 11 et contient Java 11, Hadoop 3.3.5, Spark 3.4.3 et Python 3. La commande de construction est :

> *docker build -t hadoop-base:3.3.5 base/*

**afin de construire l image a partir du docker file .**

Dockerfile.txt

Fichier Modifier Affichage

```
FROM debian:11

RUN apt-get update && apt-get install -y --no-install-recommends \
    openjdk-11-jdk \
    net-tools \
    curl \
    netcat \
    gnupg \
    libsnappy-dev \
    wget && \
    rm -rf /var/lib/apt/lists/*

RUN apt-get update && apt-get install -y python3 python3-setuptools python3-pip

ENV JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64/

RUN curl -O https://dist.apache.org/repos/dist/release/hadoop/common/KEYS
RUN gpg --import KEYS

ENV HADOOP_VERSION=3.3.5
ENV HADOOP_URL=https://www.apache.org/dist/hadoop/common/hadoop-$HADOOP_VERSION/hadoop-$HADOOP_VERSION.tar.gz

RUN set -x \
    && curl -fSL "$HADOOP_URL" -o /tmp/hadoop.tar.gz \
    && tar -xvf /tmp/hadoop.tar.gz -C /opt/ \
    && rm /tmp/hadoop.tar.gz

RUN ln -s /opt/hadoop-$HADOOP_VERSION/etc/hadoop /etc/hadoop
RUN mkdir /opt/hadoop-$HADOOP_VERSION/logs
RUN mkdir /hadoop-data

ENV HADOOP_HOME=/opt/hadoop-$HADOOP_VERSION
ENV HADOOP_CONF_DIR=/etc/hadoop
ENV MULTIHOMED_NETWORK=1
ENV USER=root
ENV PATH=$HADOOP_HOME/bin/:$PATH

ENV SPARK_VERSION=3.4.3
ENV SPARK_URL=https://downloads.apache.org/spark/$SPARK_VERSION/$SPARK_VERSION-bin-hadoop3.tgz
ENV SPARK_HOME=/opt/$SPARK_VERSION-bin-hadoop3
ENV PATH=$SPARK_HOME/bin:$PATH
ENV PYSPARK_PYTHON=python3
ENV PYTHONHASHSEED=1

RUN set -x \
    && curl -fSL "$SPARK_URL" -o /tmp/spark.tar.gz \
    && tar -xvzf /tmp/spark.tar.gz -C /opt/ \
    && rm /tmp/spark.tar.gz

ADD conf/core-site.xml $HADOOP_CONF_DIR/core-site.xml
ADD conf/hdfs-site.xml $HADOOP_CONF_DIR/hdfs-site.xml
ADD conf/mapred-site.xml $HADOOP_CONF_DIR/mapred-site.xml
ADD conf/yarn-site.xml $HADOOP_CONF_DIR/yarn-site.xml
ADD conf/core-site.xml $SPARK_HOME/conf/core-site.xml
ADD conf/yarn-site.xml $SPARK_HOME/conf/yarn-site.xml

ADD entrypoint.sh /entrypoint.sh
RUN chmod +x /entrypoint.sh
ENTRYPOINT ["/entrypoint.sh"]
```

☑ *Image construite avec succès ! Toutes les dépendances (Java, Hadoop, Spark, Python) sont intégrées dans une seule image réutilisable*

***Le Dockerfile commence par une image Debian 11 vierge (`FROM debian:11`), puis installe les outils système nécessaires comme Java 11 (`apt-get install openjdk-11-jdk`), indispensable car Hadoop tourne sur la JVM, ainsi que Python 3 pour PySpark. Ensuite, Hadoop 3.3.5 et Spark 3.4.3 sont téléchargés directement depuis les serveurs officiels Apache (`curl -fSL "$HADOOP_URL"`) et extraits manuellement dans le répertoire `/opt/`. Les variables d'environnement sont ensuite définies (`JAVA_HOME`, `HADOOP_HOME`, `SPARK_HOME`) pour que le système sache où trouver ces outils. Les fichiers de configuration XML (`core-site.xml`, `hdfs-site.xml`, `mapred-site.xml`, `yarn-site.xml`) sont copiés dans le conteneur pour paramétrer le cluster. Enfin, un script `entrypoint.sh` est ajouté (`ENTRYPOINT ["/entrypoint.sh"]`) pour démarrer automatiquement le service approprié au lancement du conteneur.***



## 1.5 Étapes 5 à 8 — Construction des services du cluster

Chaque service du cluster possède son propre Dockerfile, hérité de l'image de base. Voici le récapitulatif des services et leurs rôles respectifs :

| Service | Port(s) | Rôle |
|---|---|---|
| **Namenode** | 9870, 9000 | Cerveau de HDFS — gère la table des matières du système de fichiers distribué |
| **Datanode-1/2/3** | 9864/65/66 | Disques durs du cluster — stockent physiquement les blocs de données (réplication ×3) |
| **ResourceManager** | 8088, 8032 | Chef d'orchestre YARN — alloue les ressources CPU/RAM aux jobs |
| **NodeManager** | 8042 | Agent YARN local — exécute concrètement les tâches sur chaque nœud |
| **HistoryServer** | 8188 | Conserve les logs et statistiques des jobs terminés |
| **Spark Master** | 8081, 7077 | Coordinateur Spark — distribue les tâches aux workers |
| **Spark Worker-1/2** | 8082, 8083 | Exécuteurs Spark — traitent les données en mémoire RAM |

```
FROM hadoop-base:3.3.5

HEALTHCHECK CMD curl -f http://localhost:9870/ || exit 1

ENV HDFS_CONF_dfs_namenode_name_dir=file:///hadoop/dfs/name
RUN mkdir -p /hadoop/dfs/name
VOLUME /hadoop/dfs/name

ADD run.sh /run.sh
RUN chmod a+x /run.sh

EXPOSE 9870 9000
CMD ["/run.sh"]
'@ | Out-File -FilePath "C:\Users\User\Desktop\hadoop-projet\namenode\Dockerfile" -Encoding utf8 -NoNewline
PS C:\Users\User\Desktop\hadoop-projet>
```

```
PS C:\Users\User\Desktop\hadoop-projet> @'
>> FROM hadoop-base:3.3.5
>>
>> HEALTHCHECK CMD curl -f http://localhost:9864/ || exit 1
>>
>> ENV HDFS_CONF_dfs_datanode_data_dir=file:///hadoop/dfs/data
>> RUN mkdir -p /hadoop/dfs/data
>> VOLUME /hadoop/dfs/data
>>
>> ADD run.sh /run.sh
>> RUN chmod a+x /run.sh
>>
>> EXPOSE 9864
>> CMD ["/run.sh"]
>> '@ | Out-File -FilePath "C:\Users\User\Desktop\hadoop-projet\datanode\Dockerfile" -Encoding utf8 -NoNewline
PS C:\Users\User\Desktop\hadoop-projet>
```

```
PS C:\Users\User\Desktop\hadoop-projet> @'
>> FROM hadoop-base:3.3.5
>>
>> HEALTHCHECK CMD curl -f http://localhost:8088/ || exit 1
>>
>> ADD run.sh /run.sh
>> RUN chmod a+x /run.sh
>>
>> EXPOSE 8088 8032
>> CMD ["/run.sh"]
>> '@ | Out-File -FilePath "C:\Users\User\Desktop\hadoop-projet\resourcemanager\Dockerfile" -Encoding utf8 -NoNewline
PS C:\Users\User\Desktop\hadoop-projet>
```

```
PS C:\Users\User\Desktop\hadoop-projet> @'
>> FROM hadoop-base:3.3.5
>>
>> HEALTHCHECK CMD curl -f http://localhost:8042/ || exit 1
>>
>> ADD run.sh /run.sh
>> RUN chmod a+x /run.sh
>>
>> EXPOSE 8042
>> CMD ["/run.sh"]
>> '@ | Out-File -FilePath "C:\Users\User\Desktop\hadoop-projet\nodemanager\Dockerfile" -Encoding utf8 -
NoNewline
PS C:\Users\User\Desktop\hadoop-projet>
```

```
PS C:\Users\User\Desktop\hadoop-projet> @'
>> FROM hadoop-base:3.3.5
>>
>> HEALTHCHECK CMD curl -f http://localhost:8188/ || exit 1
>>
>> ENV YARN_CONF_yarn_timeline__service_leveldb__timeline__store_path=/hadoop/yarn/timeline
>> RUN mkdir -p /hadoop/yarn/timeline
>> VOLUME /hadoop/yarn/timeline
>>
>> ADD run.sh /run.sh
>> RUN chmod a+x /run.sh
>>
>> EXPOSE 8188
>> CMD ["/run.sh"]
>> '@ | Out-File -FilePath "C:\Users\User\Desktop\hadoop-projet\historyserver\Dockerfile" -Encoding utf8
-NoNewline
```

```
PS C:\Users\User\Desktop\hadoop-projet> @'
>> FROM hadoop-base:3.3.5
>>
>> ENV SPARK_MASTER_PORT=7077
>> ENV SPARK_MASTER_WEBUI_PORT=8081
>> ENV SPARK_WORKER_WEBUI_PORT=8081
>> ENV SPARK_MASTER="spark://spark-master:7077"
>>
>> RUN mkdir -p /opt/spark-3.4.3-bin-hadoop3/logs
>> RUN pip install pyspark
>>
>> ADD start-spark.sh /start-spark.sh
>> RUN chmod a+x /start-spark.sh
>>
>> EXPOSE 8081 7077
>> CMD ["/start-spark.sh"]
>> '@ | Out-File -FilePath "C:\Users\User\Desktop\hadoop-projet\spark\Dockerfile" -Encoding utf8 -NoNe
ine
PS C:\Users\User\Desktop\hadoop-projet>
```

## 1.6 Étape 9 — Lancement du cluster avec docker-compose

Le fichier docker-compose.yml orchestre tous les services. Une seule commande suffit pour démarrer l'intégralité du cluster :

*docker-compose up –d*

```
PS C:\Users\User\Desktop\hadoop-projet> docker-compose up -d
[+] up 10/10
 ✔Container spark-master      Running
 ✔Container namenode          Running
 ✔Container spark-worker-2    Running
 ✔Container datanode-1        Running
 ✔Container datanode-2        Running
 ✔Container spark-worker-1    Running
 ✔Container datanode-3        Running
 ✔Container historyserver     Running
 ✔Container nodemanager       Running
 ✔Container resourcemanager   Running
PS C:\Users\User\Desktop\hadoop-projet> docker ps
```

Vérification que tous les conteneurs sont bien démarrés :

*docker ps*

```
PS C:\Users\User\Desktop\hadoop-projet> docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED      STATUS                            PORTS                                                
```

# 2-Application pratique — TP MapReduce

Avant l'analyse COVID-19, un exercice pratique MapReduce est réalisé sur le dataset purchases.txt pour valider le bon fonctionnement du cluster.

## Étape 1 — Transfert des données dans HDFS

On crée un répertoire /data dans HDFS et on y transfère le fichier purchases.txt :

```
root@ad15ea19a059:/# hdfs dfs -mkdir -p /data
hdfs dfs -put /tmp/purchases.txt /data/
hdfs dfs -ls /data/
```

## Étape 2 — Lancement du job MapReduce

*Le mapper extrait les paires (store, item) depuis chaque ligne. Le reducer agrège les résultats et détermine l'article le plus vendu par magasin.*

Mapper.py :

```
#!/usr/bin/python3
import sys

for line in sys.stdin:
    data = line.strip().split("\t")
    if len(data) == 6:
        date, time, store, item, cost, payment = data
        print(f"{store}\t{item}\t1")
```

Reducer.py

```
#!/usr/bin/python3
import sys

oldStore = None
oldItem = None
totalCount = 0
maxItem = None
maxItemCount = 0
maxIndex = 0
listItems = []
listCount = []

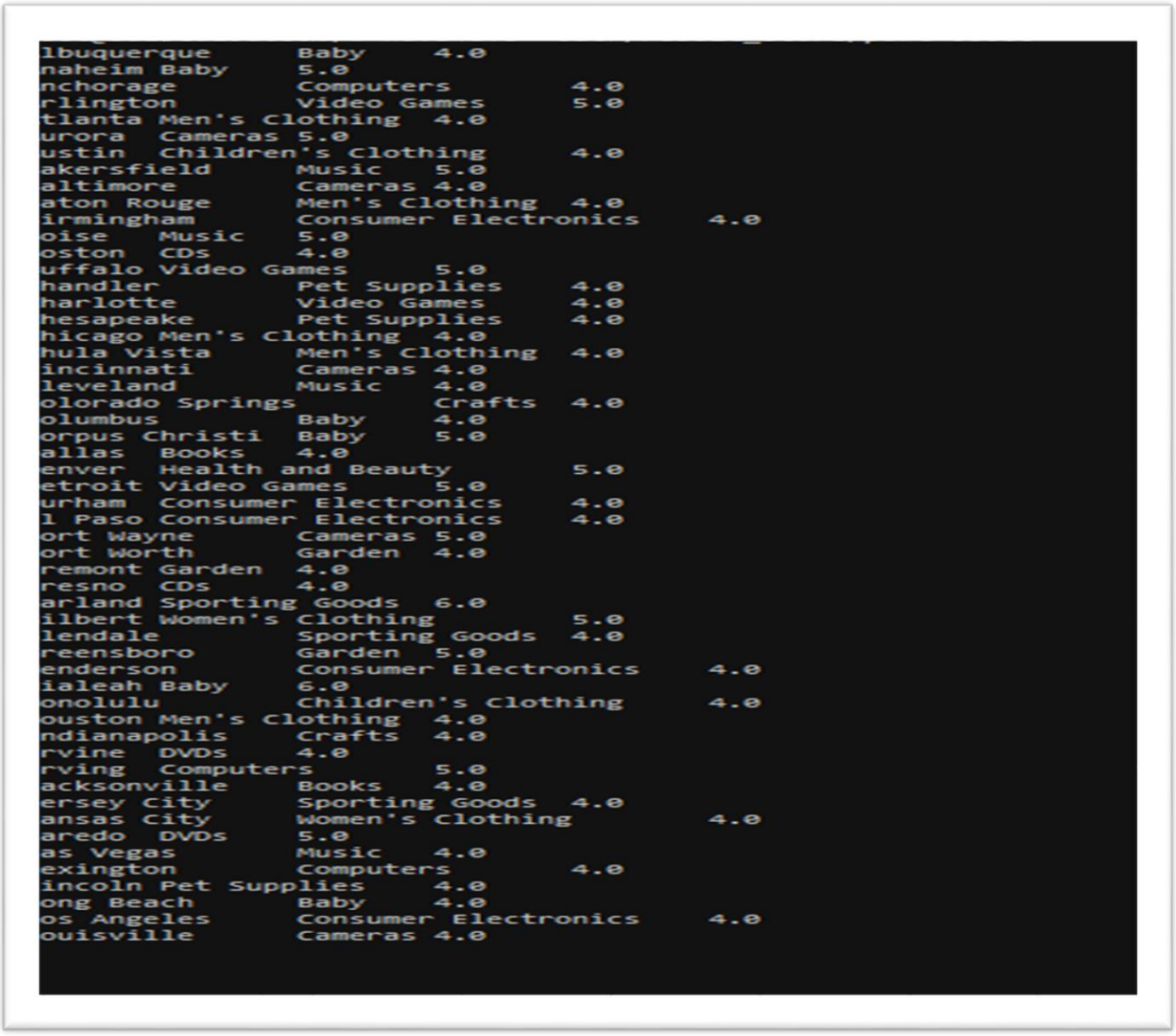
for line in sys.stdin:
    data_mapped = line.strip().split("\t")
    if len(data_mapped) != 3:
        continue
    thisStore, thisItem, thisCount = data_mapped
    if oldStore and oldStore == thisStore:
        if oldItem and oldItem != thisItem:
            listItems.append(oldItem)
            listCount.append(totalCount)
            totalCount = 0
    elif oldStore and oldStore != thisStore:
        listItems.append(thisItem)
        listCount.append(totalCount)
        maxItemCount = max(listCount)
        maxIndex = listCount.index(maxItemCount)
        maxItem = listItems[maxIndex]
        print(f"{oldStore}\t{maxItem}\t{maxItemCount}")
        listCount = []
        listItems = []
        totalCount = 0
    oldStore = thisStore
    oldItem = thisItem
    totalCount += float(thisCount)

if oldStore != None:
    print(f"{oldStore}\t{maxItem}\t{maxItemCount}")
```

Le job MapReduce est lancé via la commande hadoop jar en utilisant Hadoop Streaming, qui permet d'utiliser des scripts Python comme mapper et reducer :

```
root@ad15ea19a059:/# hadoop jar /opt/hadoop-3.3.5/share/hadoop/tools/lib/hadoop-streaming-3.3.5.jar \
-files /tmp/mapper_store.py,/tmp/reducer_store.py \
-mapper "python3 mapper_store.py" \
-reducer "python3 reducer_store.py" \
-input /data/purchases.txt \
-output /result_store
```

*Resultat obtenue :*




Chaque ligne représente donc **l'article le plus vendu dans une ville donnée**, calculé à partir de l'ensemble des transactions du fichier `purchases.txt`.

### *Ce que ce résultat démontre*

Le pipeline MapReduce a traité **4 138 476 lignes** de données de manière distribuée sur le cluster Hadoop, réparties en **2 splits** traités en parallèle par les Mappers. Le Reducer a ensuite agrégé les résultats pour produire **103 résultats finaux**.

- Le **Mapper** a extrait de chaque ligne la paire `(ville, article)`, en ignorant les lignes mal formées
- Le **Reducer** a regroupé toutes les ventes par ville, compté les occurrences de chaque article, et retenu uniquement **l'article le plus fréquent**

Par exemple, la ville de **Garland** enregistre le score le plus élevé avec **6 ventes** de Sporting Goods, tandis que la majorité des villes atteignent **4 à 5 ventes** pour leur article dominant.

# 3. Présentation du Dataset COVID-19

Le dataset est publié sur Kaggle sous le titre « Corona Virus Report » par l'utilisateur imdevskp. Les données proviennent du Centre de Science et d'Ingénierie des Systèmes de l'Université Johns Hopkins, disponibles publiquement sur GitHub.

> *Fichier principal : country_wise_latest.csv*
> *187 lignes (une par pays/territoire) et 15 colonnes*

## 3.1 Description des colonnes

| # | Colonne | Type | Description |
|---|---|---|---|
| **1** | Country/Region | String | Nom du pays — clé d'identification principale |
| **2** | Confirmed | Integer | Nombre total de cas confirmés |
| **3** | Deaths | Integer | Nombre total de décès COVID-19 |
| **4** | Recovered | Integer | Nombre total de guérisons |
| **5** | Active | Integer | Cas actifs = Confirmés − Décès − Guéris |
| **6** | New cases | Integer | Nouveaux cas (dernière mise à jour) |
| **7** | New deaths | Integer | Nouveaux décès (dernière mise à jour) |
| **8** | New recovered | Integer | Nouvelles guérisons (dernière mise à jour) |
| **9** | Deaths/100 Cases | Float | KPI 1 — Taux de létalité (%) |
| **10** | Recovered/100 Cases | Float | KPI 2 — Taux de guérison (%) |
| **11** | Deaths/100 Recovered | Float | Ratio décès/guéris — indicateur complémentaire |
| **12** | Confirmed last week | Integer | Cas confirmés il y a 7 jours |
| **13** | 1 week change | Integer | Variation absolue sur 7 jours |
| **14** | 1 week % increase | Float | Taux de croissance hebdomadaire (%) |
| **15** | WHO Region | String | Région OMS — analyses par zone géographique |

## 3.2-Pourquoi ce dataset:

Le dataset **« Corona Virus Report »** de Kaggle (imdevskp) a été choisi pour sa **fiabilité** — les données proviennent de l'Université Johns Hopkins — et sa **complétude** avec 187 pays et 15 colonnes couvrant cas confirmés, décès, guérisons et région OMS. Son format **CSV structuré** est parfaitement compatible avec HDFS et PySpark. Les données représentent un snapshot de la situation mondiale en **juillet/août 2020**, avant la vaccination. Seule limite notable : les pays avec peu de moyens de dépistage peuvent présenter des guérisons **sous-estimées**.

# 4. Définition des Indicateurs Clés de Performance (KPIs)

Un KPI (Key Performance Indicator) est une mesure chiffrée permettant d'évaluer l'état d'une situation par rapport à un objectif. Dans le contexte de l'analyse COVID-19, deux KPIs principaux ont été définis et calculés avec Apache Spark.

**KPI 1 — Taux de Mortalité**

***Formule : (Décès / Cas confirmés) × 100***

Mesure le % de patients décédés parmi les cas confirmés. Reflète la sévérité de la maladie et la capacité hospitalière.

**KPI 2 — Taux de Guérison**

***Formule : (Guéris / Cas confirmés) × 100***

Mesure le % de patients guéris parmi les cas confirmés. Évalue l'efficacité du système de santé et des protocoles de traitement.

Ces 2 KPIs ont été choisis car ils sont les indicateurs les plus fondamentaux en épidémiologie :

# 5. Chargement du Dataset dans HDFS et Exécution Spark

## 4.1 Copie du fichier CSV vers le conteneur

Le fichier CSV country_wise_latest.csv se trouve sur la machine Windows locale. Il faut le transférer dans le conteneur spark-master via la commande docker cp :

```
PS C:\Users\User\Desktop\hadoop-projet> docker cp "C:\Users\User\Desktop\hadoop-projet\Data\country_wise_latest.csv" spark-master:/tmp/
Successfully copied 16.4kB to spark-master:/tmp/
```

*Docker cp copie un fichier depuis la machine hôte Windows vers l'intérieur d'un conteneur Docker. Le fichier est ensuite accessible à l'adresse /tmp/country_wise_latest.csv dans le conteneur.*

## 5.2 Connexion au conteneur spark-master

Pour exécuter des commandes HDFS et Spark, on ouvre un terminal interactif dans le conteneur :

*À partir de cette étape, toutes les commandes sont exécutées DANS le conteneur Linux. Le prompt devient : root@e7dfd0b627af:/#*

```
PS C:\Users\User\Desktop\hadoop-projet> docker exec -it spark-master bash
root@e7dfd0b627af:/#  hdfs dfs -mkdir -p /data/covid
root@e7dfd0b627af:/# hdfs dfs -put /tmp/country_wise_latest.csv /data/covid/
root@e7dfd0b627af:/# hdfs dfs -ls /data/covid/
Found 1 items
-rw-r--r--   3 root supergroup      14784 2026-04-22 23:41 /data/covid/country_wise_latest.csv
root@e7dfd0b627af:/#
```

## 5.3 Lancement du script PySpark sur le cluster

```
PS C:\Users\User\Desktop\hadoop-projet> docker exec -it spark-master bash
root@42cc554f5439:/# pyspark --master spark://spark-master:7077
/opt/spark-3.4.3-bin-hadoop3/bin/load-spark-env.sh: line 68: ps: command not found
/opt/spark-3.4.3-bin-hadoop3/bin/load-spark-env.sh: line 68: ps: command not found
Python 3.9.2 (default, Apr 14 2026, 04:38:32)
[GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
/opt/spark-3.4.3-bin-hadoop3/bin/load-spark-env.sh: line 68: ps: command not found
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/05/01 21:34:33 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /__ / .__/\_,_/_/ /_/\_\   version 3.4.3
      /_/

Using Python version 3.9.2 (default, Apr 14 2026 04:38:32)
Spark context Web UI available at http://42cc554f5439:4040
Spark context available as 'sc' (master = spark://spark-master:7077, app id = app-20260501213435-0000).
SparkSession available as 'spark'.
```

## 5.4. Charger le CSV depuis HDFS et Créer la vue SQL

```
SparkSession available as 'spark'.
>>> Vdf = spark.read.option("header", "true").option("inferSchema", "true").csv("hdfs://namenode:9000/da
ta/covid/country_wise_latest.csv")
>>> df.createOrReplaceTempView("covid")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'df' is not defined
>>> Vdf.createOrReplaceTempView("covid")
```

Le script d'analyse est créé directement dans le conteneur et lancé via spark-submit, qui soumet le job au Spark Master

# 6. Analyse Descriptive avec PySpark

Dans ce travail, les données COVID-19 sont exploitées avec  (PySpark) pour effectuer une analyse complète : statistiques descriptives, calcul des KPIs, puis analyse comparative par pays et par région OMS.

## 6.1 Statistiques descriptives globales

L'analyse descriptive porte sur 187 pays. Les écarts-types élevés révèlent une forte disparité entre les nations : certains pays concentrent la grande majorité des cas mondiaux.

```
>>> df = spark.read.option("header", "true").option("inferSchema", "true").csv("hdfs://namenode:9000/data/covid/country_wise_latest.csv")
>>> df.select("Confirmed", "Deaths", "Recovered", "Active").describe().show()
26/05/01 22:18:03 WARN package: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.
+-------+-----------------+-----------------+-----------------+-----------------+
|summary|        Confirmed|           Deaths|        Recovered|           Active|
+-------+-----------------+-----------------+-----------------+-----------------+
|  count|              187|              187|              187|              187|
|   mean|  88130.935828877| 3497.51871657754| 50631.48128342246|  34001.935828877|
| stddev|383318.6638306154|14100.00248201848|190188.18964313966|213326.17337142891|
|    min|               10|                0|                0|                0|
|    max|          4290259|           148011|          1846641|          2816444|
+-------+-----------------+-----------------+-----------------+-----------------+
```

| Statistique | Cas confirmés | Décès | Guéris | Actifs |
|---|---|---|---|---|
| **Nombre de pays** | 187 | 187 | 187 | 187 |
| **Moyenne** | 88 131 | 3 498 | 50 631 | 34 002 |
| **Écart-type** | 383 319 | 14 100 | 190 188 | 213 326 |
| **Minimum** | 10 | 0 | 0 | 0 |
| **Maximum** | 4 290 259 | 148 011 | 1 846 641 | 2 816 444 |

### 6.2-KPI 1 — Taux de mortalité

Le KPI 1 varie fortement selon les pays. Le Yémen affiche le taux le plus élevé (28,56 %), ce qui reflète soit une situation sanitaire critique, soit un biais de données lié à la sous-estimation des cas

confirmés. Les pays européens comme la Belgique (14,79 %), l'Italie (14,26 %) et la France (13,71 %) affichent également des taux élevés.

```
>>> spark.sql("select `Country/Region`, Confirmed, Deaths, round((Deaths / Confirmed) * 100, 2) as KPI1_Mortalite_pct from covid order by KPI1_Mortalite_pct desc").show(15)
+--------------+---------+------+-----------------+
|Country/Region|Confirmed|Deaths|KPI1_Mortalite_pct|
+--------------+---------+------+-----------------+
|         Yemen|     1691|   483|            28.56|
|United Kingdom|   301708| 45844|            15.19|
|       Belgium|    66428|  9822|            14.79|
|         Italy|   246286| 35112|            14.26|
|        France|   220352| 30212|            13.71|
|       Hungary|     4448|   596|             13.4|
|   Netherlands|    53413|  6160|            11.53|
|        Mexico|   395489| 44022|            11.13|
|         Spain|   272421| 28432|            10.44|
|Western Sahara|       10|     1|             10.0|
|          Chad|      922|    75|             8.13|
|        Canada|   116458|  8944|             7.68|
|        Sweden|    79395|  5700|             7.18|
|       Ecuador|    81161|  5532|             6.82|
|       Ireland|    25892|  1764|             6.81|
+--------------+---------+------+-----------------+
only showing top 15 rows
```

## 6.3-KPI 2 — Taux de guérison

Le KPI 2 est particulièrement élevé pour Djibouti (98,38 %), l'Islande (98,33 %) et le Qatar (97,02 %), reflétant une excellente prise en charge médicale .

```
>>> spark.sql("select `Country/Region`, Confirmed, Recovered, round((Recovered / Confirmed) * 100, 2) as KPI2_Guerison_pct from covid order by KPI2_Guerison_pct desc").show(15)
+--------------+---------+---------+-----------------+
|Country/Region|Confirmed|Recovered|KPI2_Guerison_pct|
+--------------+---------+---------+-----------------+
|      Dominica|       18|       18|            100.0|
|       Grenada|       23|       23|            100.0|
|      Holy See|       12|       12|            100.0|
|      Djibouti|     5059|     4977|            98.38|
|       Iceland|     1854|     1823|            98.33|
|        Brunei|      141|      138|            97.87|
|   New Zealand|     1557|     1514|            97.24|
|         Qatar|   109597|   106328|            97.02|
|      Malaysia|     8904|     8601|             96.6|
|     Mauritius|      344|      332|            96.51|
|        Norway|     9132|     8752|            95.84|
|       Taiwan*|      462|      440|            95.24|
|          Laos|       20|       19|             95.0|
|         Malta|      701|      665|            94.86|
|       Estonia|     2034|     1923|            94.54|
+--------------+---------+---------+-----------------+
only showing top 15 rows
```

# 7. Analyse Prescriptive

L'analyse prescriptive : elle catégorise chaque pays selon ses KPIs et formule des recommandations d'action concrètes à destination des décideurs sanitaires. C'est l'étape où la donnée se transforme en aide à la décision.

## 7.1-Résultats

La Situation modérée regroupe le plus grand nombre de pays (55 pays, soit 29,4%), suivie de près par la Vigilance requise (54 pays, 28,9%). La Situation contrôlée concerne 49 pays (26,2%), ce qui représente environ un quart des pays ayant réussi à maîtriser la pandémie. Enfin, seulement 29 pays (15,5%) sont en Intervention urgente, mais ce sont les plus critiques car ils concentrent les taux de mortalité les plus élevés.

```
>>> spark.sql("select Categorie, count(*) as Nb_Pays, round(avg(KPI1),2) as Moy_Mortalite, round(min(KPI1),2) as Min_Mortalite, round(max(KPI1),2) as Max_Mortalite, round(avg(KPI2),2) as Moy_Guerison from (selec
t `Country/Region`, round((Deaths/Confirmed)*100,2) as KPI1, round((Recovered/Confirmed)*100,2) as KPI2, case when (Deaths/Confirmed)*100 >= 5 then 'Intervention urgente' when (Deaths/Confirmed)*100 >= 2 and (Re
covered/Confirmed)*100 >= 50 then 'Vigilance requise' when (Deaths/Confirmed)*100 < 2 and (Recovered/Confirmed)*100 > 70 then 'Situation controlee' else 'Situation moderee' end as Categorie from covid) group by
Categorie order by Moy_Mortalite desc").show()
+--------------------+-------+--------------+-------------+-------------+-----------+
|           Categorie|Nb_Pays|Moy_Mortalite|Min_Mortalite|Max_Mortalite|Moy_Guerison|
+--------------------+-------+--------------+-------------+-------------+-----------+
|Intervention urgente|     29|          8.96|         5.03|        28.56|      58.24|
|   Vigilance requise|     54|           3.3|         2.06|         4.93|      74.29|
|   Situation moderee|     55|          1.61|          0.0|         4.78|      39.84|
| Situation controlee|     49|          0.78|          0.0|         1.98|      86.32|
+--------------------+-------+--------------+-------------+-------------+-----------+
```

## 7.2 Resultat

🔴 **Intervention urgente** : avec une mortalité moyenne de **8,96%** et un pic à **28,56%** , ces 29 pays sont dans une situation critique. Malgré cela, leur taux de guérison moyen de **58,24%** reste moyen, ce qui s'explique par le fait que certains pays dans cette catégorie comme l'Italie (80,64%) et l'Irlande (90,24%) ont quand même bien géré les guérisons malgré une mortalité élevée due au choc initial de la pandémie.

🟠 **Vigilance requise** : la mortalité moyenne de **3,30%** reste préoccupante mais maîtrisable. Le taux de guérison de **74,29%** est le deuxième meilleur, ce qui montre que ces pays ont des systèmes de santé fonctionnels mais sous pression.

🟡 **Situation modérée** : Ces 55 pays ont une mortalité faible (**1,61%**) mais un taux de guérison très bas de **39,84%**. Cela ne signifie pas une mauvaise gestion sanitaire, mais plutôt un **problème de reporting** — beaucoup de ces pays ne disposaient pas des moyens pour suivre et comptabiliser toutes les guérisons.

**Situation contrôlée** : ces 49 pays affichent les meilleurs résultats avec une mortalité quasi nulle (**0,78%**) et un taux de guérison excellent de **86,32%**, preuve d'une gestion rigoureuse et d'un système de santé performant.

| Country/Region | KPI1 | KPI2 | Categorie |
|---|---|---|---|
| Yemen | 28.56 | 49.26 | Intervention urgente |
| United Kingdom | 15.19 | 0.48 | Intervention urgente |
| Belgium | 14.79 | 26.27 | Intervention urgente |
| Italy | 14.26 | 80.64 | Intervention urgente |
| France | 13.71 | 36.86 | Intervention urgente |
| Hungary | 13.4 | 74.84 | Intervention urgente |
| Netherlands | 11.53 | 0.35 | Intervention urgente |
| Mexico | 11.13 | 76.82 | Intervention urgente |
| Spain | 10.44 | 55.2 | Intervention urgente |
| Western Sahara | 10.0 | 80.0 | Intervention urgente |
| Chad | 8.13 | 87.85 | Intervention urgente |
| Canada | 7.68 | 0.0 | Intervention urgente |
| Sweden | 7.18 | 0.0 | Intervention urgente |
| Ecuador | 6.82 | 43.0 | Intervention urgente |
| Ireland | 6.81 | 90.24 | Intervention urgente |
| Barbados | 6.36 | 85.45 | Intervention urgente |
| Sudan | 6.3 | 51.99 | Intervention urgente |
| Liberia | 6.17 | 55.36 | Intervention urgente |
| Niger | 6.1 | 90.72 | Intervention urgente |
| San Marino | 6.01 | 93.99 | Intervention urgente |
| Syria | 5.93 | 0.0 | Intervention urgente |
| Switzerland | 5.74 | 89.62 | Intervention urgente |
| Andorra | 5.73 | 88.53 | Intervention urgente |
| Slovenia | 5.56 | 83.04 | Intervention urgente |
| Iran | 5.42 | 86.9 | Intervention urgente |
| Trinidad and Tobago | 5.41 | 86.49 | Intervention urgente |
| China | 5.37 | 90.88 | Intervention urgente |
| Guyana | 5.14 | 46.53 | Intervention urgente |
| Egypt | 5.03 | 37.67 | Intervention urgente |

| | | | |
|---|---|---|---|
| Comoros | 1.98 | 92.66 | Situation controlee |
| Armenia | 1.9 | 71.32 | Situation controlee |
| Cyprus | 1.79 | 80.38 | Situation controlee |
| Luxembourg | 1.77 | 76.33 | Situation controlee |
| Thailand | 1.76 | 94.36 | Situation controlee |
| Burma | 1.71 | 83.43 | Situation controlee |
| Russia | 1.63 | 73.74 | Situation controlee |
| Sao Tome and Prin... | 1.62 | 84.86 | Situation controlee |
| Taiwan* | 1.52 | 95.24 | Situation controlee |
| Morocco | 1.51 | 79.25 | Situation controlee |
| Georgia | 1.41 | 81.09 | Situation controlee |
| New Zealand | 1.41 | 97.24 | Situation controlee |
| Azerbaijan | 1.39 | 76.34 | Situation controlee |
| Malaysia | 1.39 | 96.6 | Situation controlee |
| Malta | 1.28 | 94.86 | Situation controlee |
| Slovakia | 1.28 | 74.09 | Situation controlee |
| Jamaica | 1.17 | 83.7 | Situation controlee |
| Liechtenstein | 1.16 | 94.19 | Situation controlee |
| Djibouti | 1.15 | 98.38 | Situation controlee |
| Saudi Arabia | 1.03 | 82.9 | Situation controlee |
| Jordan | 0.94 | 88.52 | Situation controlee |
| Tajikistan | 0.83 | 83.32 | Situation controlee |
| Belarus | 0.8 | 89.95 | Situation controlee |
| Kuwait | 0.68 | 85.52 | Situation controlee |
| Guinea | 0.64 | 88.69 | Situation controlee |
| United Arab Emirates | 0.58 | 88.73 | Situation controlee |
| Iceland | 0.54 | 98.33 | Situation controlee |
| Oman | 0.51 | 74.01 | Situation controlee |
| Ghana | 0.5 | 88.63 | Situation controlee |
| Maldives | 0.45 | 75.6 | Situation controlee |
| Sri Lanka | 0.39 | 75.61 | Situation controlee |
| Bahrain | 0.36 | 91.46 | Situation controlee |
| Burundi | 0.26 | 79.63 | Situation controlee |
| Nepal | 0.26 | 73.35 | Situation controlee |
| Uganda | 0.18 | 87.41 | Situation controlee |
| Qatar | 0.15 | 97.02 | Situation controlee |
| Singapore | 0.05 | 89.88 | Situation controlee |
| Bhutan | 0.0 | 86.87 | Situation controlee |
| Dominica | 0.0 | 100.0 | Situation controlee |
| Eritrea | 0.0 | 72.08 | Situation controlee |
| Greenland | 0.0 | 92.86 | Situation controlee |
| Grenada | 0.0 | 100.0 | Situation controlee |
| Holy See | 0.0 | 100.0 | Situation controlee |
| Laos | 0.0 | 95.0 | Situation controlee |
| Mongolia | 0.0 | 76.82 | Situation controlee |
| Saint Kitts and N... | 0.0 | 88.24 | Situation controlee |
| Saint Lucia | 0.0 | 91.67 | Situation controlee |
| Saint Vincent and... | 0.0 | 75.0 | Situation controlee |
| Vietnam | 0.0 | 84.69 | Situation controlee |

| | | | |
|---|---|---|---|
| Angola | 4.32 | 25.47 | Situation moderee |
| Tanzania | 4.13 | 35.95 | Situation moderee |
| Bolivia | 3.72 | 30.17 | Situation moderee |
| US | 3.45 | 30.9 | Situation moderee |
| Honduras | 2.93 | 12.68 | Situation moderee |
| Somalia | 2.91 | 48.28 | Situation moderee |
| Bahamas | 2.88 | 23.82 | Situation moderee |
| Bosnia and Herzeg... | 2.8 | 46.96 | Situation moderee |
| Malawi | 2.7 | 44.9 | Situation moderee |
| Gambia | 2.45 | 20.25 | Situation moderee |
| Lesotho | 2.38 | 25.35 | Situation moderee |
| Philippines | 2.37 | 32.24 | Situation moderee |
| Libya | 2.26 | 20.41 | Situation moderee |
| Serbia | 2.25 | 0.0 | Situation moderee |
| Nigeria | 2.09 | 44.2 | Situation moderee |
| South Sudan | 2.0 | 50.98 | Situation moderee |
| Senegal | 1.99 | 66.34 | Situation moderee |
| Benin | 1.98 | 58.53 | Situation moderee |
| Argentina | 1.83 | 43.35 | Situation moderee |
| Congo (Brazzaville) | 1.69 | 25.91 | Situation moderee |
| Dominican Republic | 1.69 | 47.08 | Situation moderee |
| Equatorial Guinea | 1.66 | 27.42 | Situation moderee |
| Suriname | 1.62 | 62.37 | Situation moderee |
| Kenya | 1.59 | 43.58 | Situation moderee |
| Ethiopia | 1.57 | 43.9 | Situation moderee |
| Montenegro | 1.56 | 27.96 | Situation moderee |
| South Africa | 1.56 | 60.75 | Situation moderee |
| Eswatini | 1.47 | 44.26 | Situation moderee |
| Guinea-Bissau | 1.33 | 41.1 | Situation moderee |
| Zimbabwe | 1.33 | 20.04 | Situation moderee |
| Bangladesh | 1.31 | 55.56 | Situation moderee |
| Lebanon | 1.31 | 44.02 | Situation moderee |
| Central African R... | 1.28 | 33.62 | Situation moderee |
| Australia | 1.09 | 60.84 | Situation moderee |
| Cabo Verde | 0.95 | 66.58 | Situation moderee |
| Paraguay | 0.95 | 63.87 | Situation moderee |
| Madagascar | 0.94 | 64.6 | Situation moderee |
| Venezuela | 0.93 | 62.29 | Situation moderee |
| Israel | 0.74 | 42.41 | Situation moderee |
| Costa Rica | 0.73 | 24.14 | Situation moderee |
| West Bank and Gaza | 0.73 | 35.33 | Situation moderee |
| Kazakhstan | 0.69 | 64.27 | Situation moderee |
| Gabon | 0.68 | 65.13 | Situation moderee |
| Mozambique | 0.65 | 0.0 | Situation moderee |
| Cote d'Ivoire | 0.61 | 66.18 | Situation moderee |
| Uzbekistan | 0.57 | 55.04 | Situation moderee |
| Namibia | 0.43 | 5.48 | Situation moderee |
| Botswana | 0.27 | 8.53 | Situation moderee |
| Rwanda | 0.27 | 51.89 | Situation moderee |
| Cambodia | 0.0 | 65.04 | Situation moderee |
| Fiji | 0.0 | 66.67 | Situation moderee |
| Papua New Guinea | 0.0 | 17.74 | Situation moderee |
| Seychelles | 0.0 | 34.21 | Situation moderee |
| Timor-Leste | 0.0 | 0.0 | Situation moderee |

| | | | |
|---|---|---|---|
| Mali | 4.93 | 76.12 | Vigilance requise |
| Burkina Faso | 4.82 | 84.18 | Vigilance requise |
| Indonesia | 4.82 | 58.0 | Vigilance requise |
| Romania | 4.81 | 56.19 | Vigilance requise |
| Peru | 4.73 | 69.93 | Vigilance requise |
| North Macedonia | 4.56 | 54.48 | Vigilance requise |
| Denmark | 4.45 | 91.6 | Vigilance requise |
| Finland | 4.45 | 93.54 | Vigilance requise |
| Germany | 4.41 | 91.89 | Vigilance requise |
| Belize | 4.17 | 54.17 | Vigilance requise |
| Algeria | 4.16 | 67.34 | Vigilance requise |
| Iraq | 3.96 | 68.52 | Vigilance requise |
| Lithuania | 3.96 | 80.24 | Vigilance requise |
| Kyrgyzstan | 3.91 | 63.69 | Vigilance requise |
| Guatemala | 3.89 | 71.63 | Vigilance requise |
| Poland | 3.86 | 75.7 | Vigilance requise |
| Sierra Leone | 3.7 | 73.86 | Vigilance requise |
| Brazil | 3.59 | 75.61 | Vigilance requise |
| Afghanistan | 3.5 | 69.49 | Vigilance requise |
| Antigua and Barbuda | 3.49 | 75.58 | Vigilance requise |
| Austria | 3.47 | 88.75 | Vigilance requise |
| Monaco | 3.45 | 89.66 | Vigilance requise |
| Cuba | 3.44 | 92.85 | Vigilance requise |
| Tunisia | 3.44 | 79.52 | Vigilance requise |
| Portugal | 3.42 | 70.33 | Vigilance requise |
| Colombia | 3.41 | 51.02 | Vigilance requise |
| Estonia | 3.39 | 94.54 | Vigilance requise |
| Bulgaria | 3.27 | 52.58 | Vigilance requise |
| Moldova | 3.23 | 69.77 | Vigilance requise |
| Japan | 3.2 | 70.55 | Vigilance requise |
| Nicaragua | 3.14 | 72.46 | Vigilance requise |
| Zambia | 3.08 | 61.84 | Vigilance requise |
| Albania | 2.95 | 56.25 | Vigilance requise |
| Mauritius | 2.91 | 96.51 | Vigilance requise |
| Uruguay | 2.91 | 79.12 | Vigilance requise |
| Croatia | 2.85 | 80.64 | Vigilance requise |
| Norway | 2.79 | 95.84 | Vigilance requise |
| El Salvador | 2.71 | 51.73 | Vigilance requise |
| Chile | 2.64 | 91.96 | Vigilance requise |
| Latvia | 2.54 | 85.73 | Vigilance requise |
| Mauritania | 2.51 | 74.95 | Vigilance requise |
| Kosovo | 2.5 | 54.32 | Vigilance requise |
| Turkey | 2.48 | 92.71 | Vigilance requise |
| Ukraine | 2.44 | 55.45 | Vigilance requise |
| Czechia | 2.4 | 73.65 | Vigilance requise |
| Congo (Kinshasa) | 2.35 | 64.45 | Vigilance requise |
| Cameroon | 2.29 | 84.97 | Vigilance requise |
| India | 2.26 | 64.26 | Vigilance requise |
| Haiti | 2.15 | 59.47 | Vigilance requise |
| Panama | 2.15 | 57.1 | Vigilance requise |
| Brunei | 2.13 | 97.87 | Vigilance requise |
| Pakistan | 2.13 | 87.87 | Vigilance requise |

### 7.3-Conclusion:

En combinant les deux tableaux, une image claire se dessine : sur 187 pays, seulement 26,2% ont réellement contrôlé la pandémie, tandis que 44,4% restent dans des zones de risque .

# 8- Création du Dashboard de Visualisation

L'analyse des données COVID-19 via PySpark produit des résultats sous forme de tableaux textuels dans le terminal. Bien que ces résultats soient complets et précis, ils restent difficiles à interpréter rapidement pour un lecteur non technique. Le dashboard permet de **transformer ces données brutes en visualisations interactives** claires et compréhensibles, facilitant ainsi la prise de décision et la communication des résultats.

## 8.1-Installation des bibliothèques

- pandas : convertit les DataFrames Spark en tableaux Python manipulables

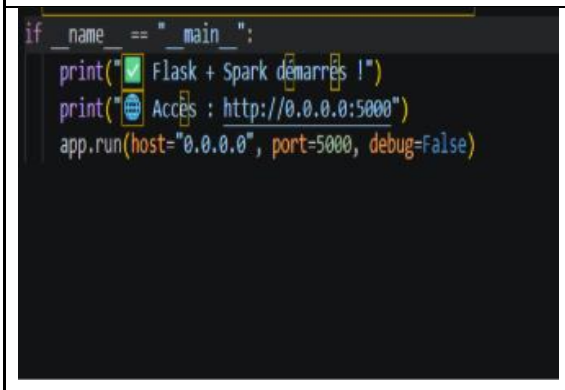
```
root@42cc554f5439:/# pip install pandas
Collecting pandas
  Downloading pandas-2.3.3-cp39-cp39-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (12.8 MB)
     |████████████████████████████████| 12.8 MB 3.2 MB/s
Collecting tzdata>=2022.7
  Downloading tzdata-2026.2-py2.py3-none-any.whl (349 kB)
     |████████████████████████████████| 349 kB 2.5 MB/s
Collecting pytz>=2020.1
  Downloading pytz-2026.1.post1-py2.py3-none-any.whl (510 kB)
     |████████████████████████████████| 510 kB 2.4 MB/s
Collecting python-dateutil>=2.8.2
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
     |████████████████████████████████| 229 kB 1.9 MB/s
Collecting numpy>=1.22.4
  Downloading numpy-2.0.2-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (19.5 MB)
     |████████████████████████████████| 19.5 MB 1.5 MB/s
Collecting six>=1.5
  Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: six, tzdata, pytz, python-dateutil, numpy, pandas
Successfully installed numpy-2.0.2 pandas-2.3.3 python-dateutil-2.9.0.post0 pytz-2026.1.post1 six-1.17.0 tzdata-2026.2
```

- **flask** : sert le dashboard dans un navigateur web

```
root@42cc554f5439:/# pip install flask
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl (103 kB)
     |████████████████████████████████| 103 kB 949 kB/s
Collecting blinker>=1.9.0
  Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting jinja2>=3.1.2
  Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
     |████████████████████████████████| 134 kB 1.8 MB/s
Collecting importlib-metadata>=3.6.0
  Downloading importlib_metadata-8.7.1-py3-none-any.whl (27 kB)
Collecting markupsafe>=2.1.1
  Downloading markupsafe-3.0.3-cp39-cp39-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (20 kB)
Collecting werkzeug>=3.1.0
  Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
     |████████████████████████████████| 226 kB 2.8 MB/s
Collecting click>=8.1.3
  Downloading click-8.1.8-py3-none-any.whl (98 kB)
     |████████████████████████████████| 98 kB 1.2 MB/s
Collecting itsdangerous>=2.2.0
  Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Collecting zipp>=3.20
  Downloading zipp-3.23.1-py3-none-any.whl (10 kB)
Installing collected packages: zipp, markupsafe, werkzeug, jinja2, itsdangerous, importlib-metadata, click, blinker, flask
Successfully installed blinker-1.9.0 click-8.1.8 flask-3.1.3 importlib-metadata-8.7.1 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8 zipp-3.23.1
```

## 8.2- Creer  le fichier Python

| | |
|---|---|
|  | **Initialisation de Spark** Dans cette partie, nous avons créé une session Spark via `SparkSession.builder` en spécifiant l'adresse du master Spark (`spark://spark-master:7077`). Cette étape est indispensable car elle établit la connexion entre l'application Flask et le cluster Spark déployé dans Docker |
|  | **Lecture des données depuis HDFS** Nous avons lu le fichier `country_wise_latest.csv` directement depuis HDFS via `spark.read.csv()` |
|  | Dans cette partie, nous avons utilisé la fonction `agg()` de PySpark pour calculer en une seule opération tous les indicateurs dont nous avons besoin : le nombre de pays, le total des cas, des décès, des guéris ainsi que les moyennes des deux KPIs. Nous avons choisi cette méthode car elle est plus rapide et plus efficace que de faire plusieurs calculs séparés sur les données. |
|  | Graphique 1 — Top 10 pays taux de mortalité (KPI 1) : |
|  | Graphique 2 — Top 10 pays taux de guérison (KPI 2) |
|  | Graphique 3 — Répartition des catégories (analyse prescriptive) : |

| | |
|---|---|
|  | Le template HTML est le code de la page web que l'utilisateur voit dans son navigateur.<br>Les données calculées par Spark sont envoyées à cette page et les graphiques sont affichés grâce à Chart.js, une bibliothèque JavaScript qui transforme les chiffres en graphiques visuels.<br>Cette approche nous a permis d'avoir tout le projet dans un seul fichier, ce qui le rend plus simple à lancer dans Docker. |
|  <br> | **Les routes Flask** Nous avons défini plusieurs routes dans l'application : la route principale `/` qui affiche le dashboard complet, et des routes API (`/api/stats`, `/api/kpi1`, `/api/kpi2`) qui retournent les données au format JSON brut. Ces routes API ont été créées pour permettre une éventuelle intégration avec d'autres outils ou tableaux de bord sans avoir à recharger toute la page. |
|  <br> | **Lancement de l'application** L'application est lancée avec `host="0.0.0.0"` pour écouter sur toutes les interfaces réseau du container, ce qui est indispensable dans un environnement Docker pour que le port soit accessible depuis la machine Windows hôte. Le port 5000 correspond à celui exposé dans le fichier `docker-compose.yml`.<br>Nous avons désactivé le mode `debug` car il est incompatible avec PySpark qui ne supporte pas le rechargement automatique des modules. |

## 8..3- Soumission et exécution du job PySpark sur le cluster



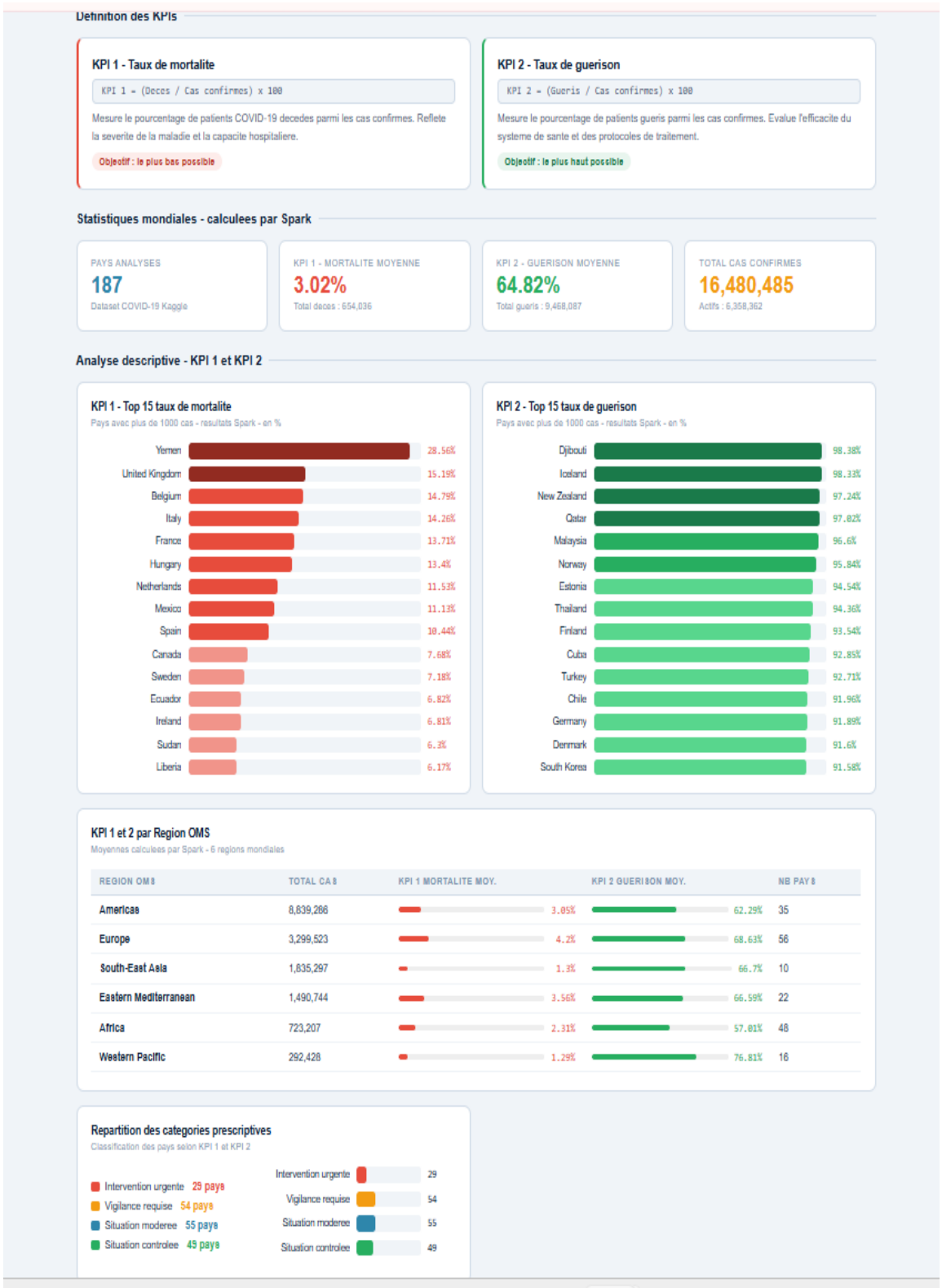

## 8.4 - Lancer l'application

Commande : pyspark master spark://spark-master:7077

*"On lance le fichier Flask — Spark se connecte automatiquement au cluster et le dashboard démarre"*

### 7.3-Le dashboard resultant Le dashboard produit trois visualisations complémentaires :

Puis on a**ccéde au dashboard** *dans le navigateur sur le port 5000"*Dans le navigateur Windows on tape http://localhost:5000


Definition des KPIs
KPI 1 - Taux de mortalite
KPI 1 = (Deces / Cas confirmes) x 100
Mesure le pourcentage de patients COVID-19 decedes parmi les cas confirmes. Reflete la severite de la maladie et la capacite hospitaliere.
Objectif : le plus bas possible
KPI 2 - Taux de guerison
KPI 2 = (Gueris / Cas confirmes) x 100
Mesure le pourcentage de patients gueris parmi les cas confirmes. Evalue l'efficacite du systeme de sante et des protocoles de traitement.
Objectif : le plus haut possible
Statistiques mondiales - calculees par Spark
PAYS ANALYSES
187
Dataset COVID-19 Kaggle
KPI 1 - MORTALITE MOYENNE
3.02%
Total deces : 654,036
KPI 2 - GUERISON MOYENNE
64.82%
Total gueris : 9,468,087
TOTAL CAS CONFIRMES
16,480,485
Actifs : 6,358,362
Analyse descriptive - KPI 1 et KPI 2
KPI 1 - Top 15 taux de mortalite
Pays avec plus de 1000 cas - resultats Spark - en %
Yemen 28.56%
United Kingdom 15.19%
Belgium 14.79%
Italy 14.26%
France 13.71%
Hungary 13.4%
Netherlands 11.53%
Mexico 11.13%
Spain 10.44%
Canada 7.68%
Sweden 7.18%
Ecuador 6.82%
Ireland 6.81%
Sudan 6.3%
Liberia 6.17%
KPI 2 - Top 15 taux de guerison
Pays avec plus de 1000 cas - resultats Spark - en %
Djibouti 98.38%
Iceland 98.33%
New Zealand 97.24%
Qatar 97.02%
Malaysia 96.6%
Norway 95.84%
Estonia 94.54%
Thailand 94.36%
Finland 93.54%
Cuba 92.85%
Turkey 92.71%
Chile 91.96%
Germany 91.89%
Denmark 91.6%
South Korea 91.58%
KPI 1 et 2 par Region OMS
Moyennes calculees par Spark - 6 regions mondiales
REGION OMS
TOTAL CAS
KPI 1 MORTALITE MOY.
KPI 2 GUERISON MOY.
NB PAYS
Americas 8,839,266 3.05% 62.29% 35
Europe 3,299,523 4.2% 68.63% 56
South-East Asia 1,835,297 1.3% 66.7% 10
Eastern Mediterranean 1,490,744 3.56% 66.59% 22
Africa 723,207 2.31% 57.01% 48
Western Pacific 292,428 1.29% 76.81% 16
Repartition des categories prescriptives
Classification des pays selon KPI 1 et KPI 2
Intervention urgente 29 pays
Vigilance requise 54 pays
Situation moderee 55 pays
Situation controlee 49 pays
Intervention urgente 29
Vigilance requise 54
Situation moderee 55
Situation controlee 49



Analyse prescriptive - Recommandations par pays

Classification et recommandations - resultats reels Spark

Basee sur KPI 1 (mortalite) et KPI 2 (guerison) calcules par PySpark sur le cluster Hadoop

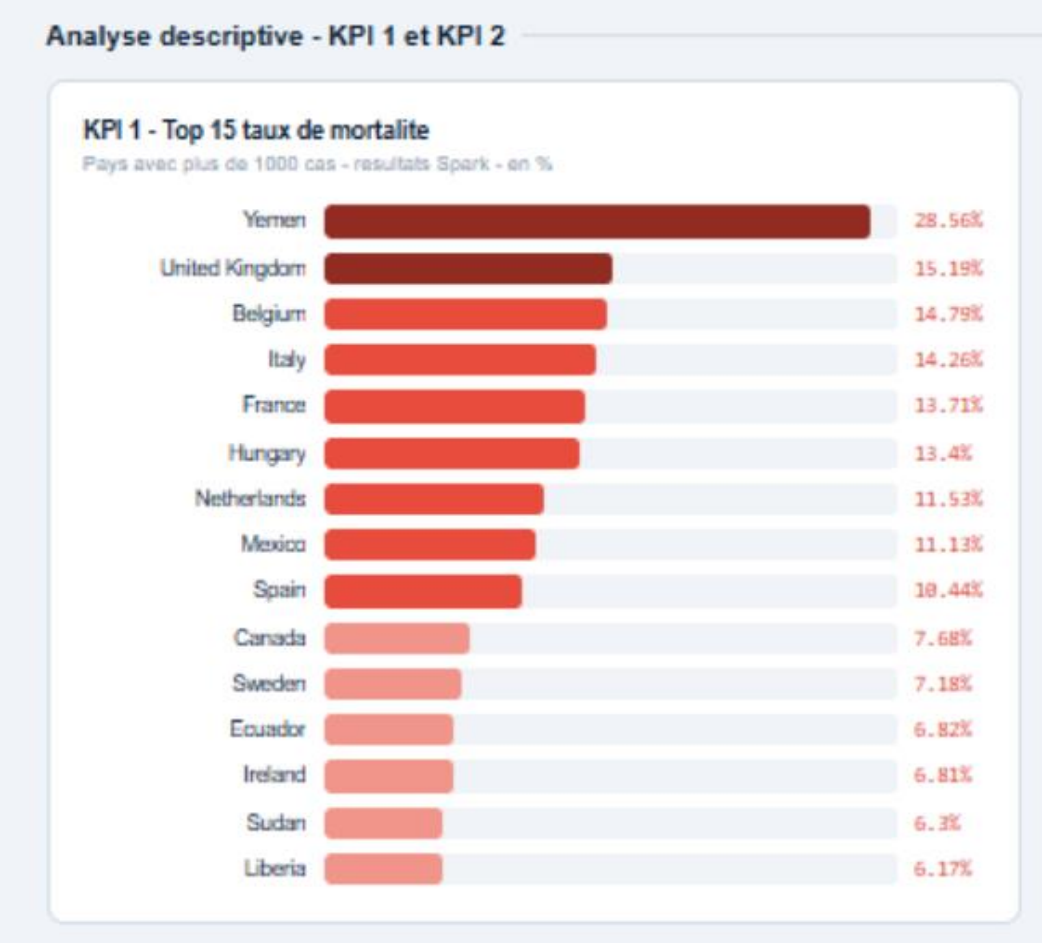
| PAYS | KPI 1 MORTALITE | KPI 2 GUERISON | CATEGORIE | RECOMMANDATION |
|---|---|---|---|---|
| Yemen | 28.56% | 49.26% | Intervention urgente | Deploiement immediat de ressources medicales |
| United Kingdom | 15.19% | 0.48% | Intervention urgente | Deploiement immediat de ressources medicales |
| Belgium | 14.79% | 26.27% | Intervention urgente | Deploiement immediat de ressources medicales |
| Italy | 14.26% | 80.64% | Intervention urgente | Deploiement immediat de ressources medicales |
| France | 13.71% | 36.86% | Intervention urgente | Deploiement immediat de ressources medicales |
| Hungary | 13.4% | 74.84% | Intervention urgente | Deploiement immediat de ressources medicales |
| Netherlands | 11.53% | 0.35% | Intervention urgente | Deploiement immediat de ressources medicales |
| Mexico | 11.13% | 76.82% | Intervention urgente | Deploiement immediat de ressources medicales |
| Spain | 10.44% | 55.2% | Intervention urgente | Deploiement immediat de ressources medicales |
| Canada | 7.68% | Non rapporte | Intervention urgente | Deploiement immediat de ressources medicales |
| Sweden | 7.18% | Non rapporte | Intervention urgente | Deploiement immediat de ressources medicales |
| Ecuador | 6.82% | 43.0% | Intervention urgente | Deploiement immediat de ressources medicales |
| Ireland | 6.81% | 90.24% | Intervention urgente | Deploiement immediat de ressources medicales |
| Sudan | 6.3% | 51.99% | Intervention urgente | Deploiement immediat de ressources medicales |
| Liberia | 6.17% | 55.36% | Intervention urgente | Deploiement immediat de ressources medicales |
| Niger | 6.1% | 90.72% | Intervention urgente | Deploiement immediat de ressources medicales |
| Switzerland | 5.74% | 89.62% | Intervention urgente | Deploiement immediat de ressources medicales |
| Slovenia | 5.56% | 83.04% | Intervention urgente | Deploiement immediat de ressources medicales |
| Iran | 5.42% | 86.9% | Intervention urgente | Deploiement immediat de ressources medicales |
| China | 5.37% | 90.88% | Intervention urgente | Deploiement immediat de ressources medicales |
| Egypt | 5.03% | 37.67% | Intervention urgente | Deploiement immediat de ressources medicales |
| Mali | 4.93% | 76.12% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |
| Burkina Faso | 4.82% | 84.18% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |
| Indonesia | 4.82% | 58.0% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |
| Romania | 4.81% | 56.19% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |
| Greece | 4.78% | 32.51% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |
| Peru | 4.73% | 69.93% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |
| North Macedonia | 4.56% | 54.48% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |
| Finland | 4.45% | 93.54% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |
| Denmark | 4.45% | 91.6% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |

## 7.5-Analyse du Dashboard COVID-19

## 1. Statistiques Mondiales Globales

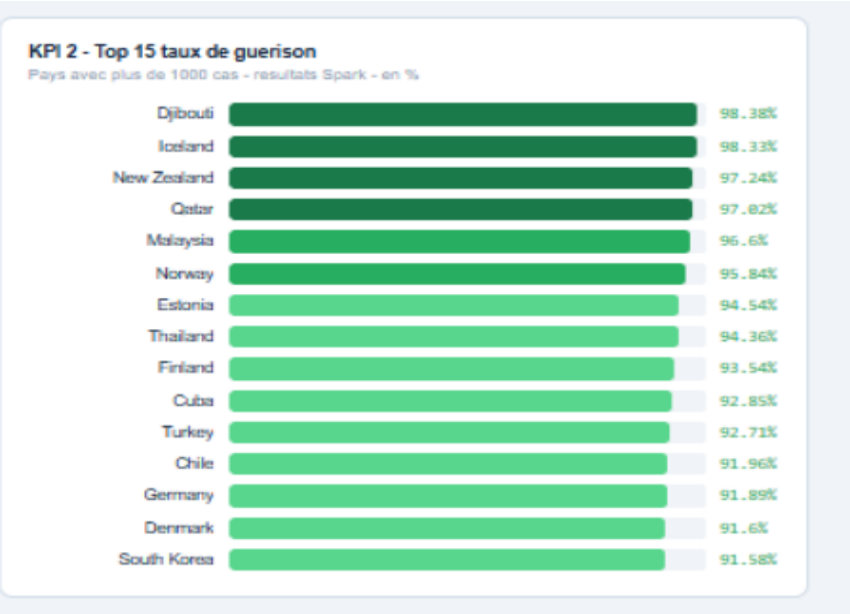
L'analyse porte sur **187 pays** issus du dataset COVID-19 Kaggle, traités par Apache Spark sur le cluster Hadoop. Les résultats globaux révèlent un taux de mortalité moyen de **3,02 %** et un taux de guérison moyen de **64,82 %**, pour un total de **16 480 485 cas confirmés** dont 6 358 362 cas encore actifs au moment de la collecte des données.

## 2. KPI 1 — Taux de Mortalité par Pays




Ce graphique montre les 15 pays avec le taux de mortalité le plus élevé. On voit clairement que le Yémen domine avec 28,56% loin devant les autres. Les pays européens comme le Royaume-Uni, la Belgique, l'Italie et la France suivent avec des taux entre 13% et 15%. Ces taux élevés s'expliquent par le fait que ces données correspondent au début de la pandémie avant la vaccination.

## 3. KPI 2 — Taux de Guérison par Pays




Les taux de guérison les plus élevés sont ceux de **Djibouti (98,38 %)**, **l'Islande (98,33 %)**, **la Nouvelle-Zélande (97,24 %)** et **le Qatar (97,02 %)**. Les pays européens comme l'**Allemagne (91,89 %)** et le **Danemark (91,6 %)** ferment le classement des 15 premiers.

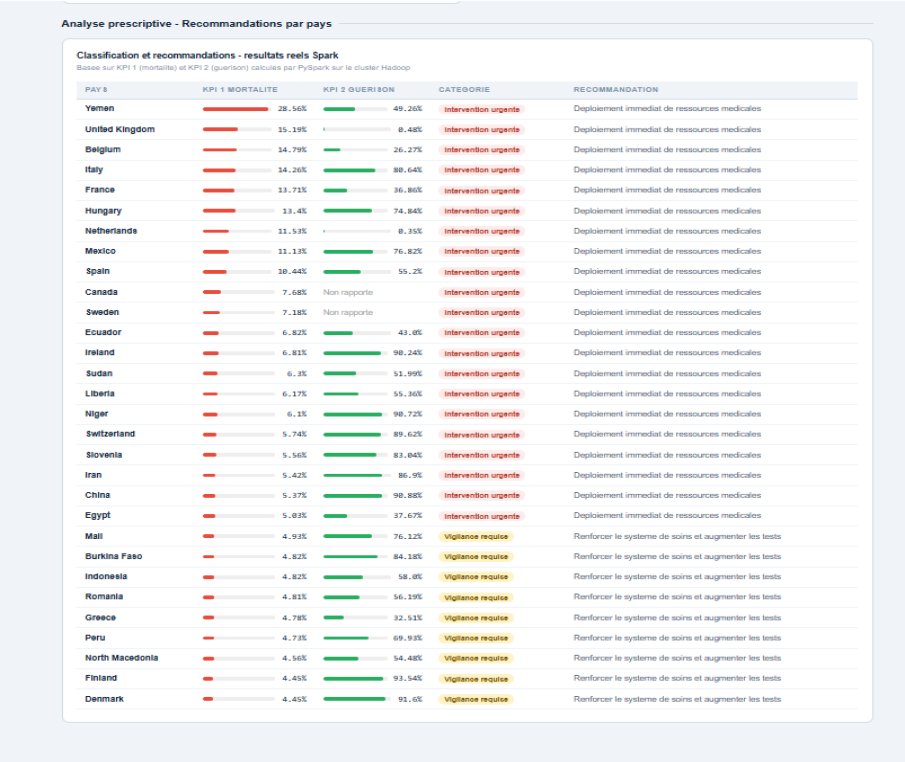
## *4. Analyse par Région OMS*




L'analyse par région OMS montre des disparités claires entre les 6 zones mondiales.

Le **Pacifique occidental** affiche les meilleurs résultats : mortalité la plus faible (1,29 %) et guérison la plus élevée (76,81 %). L'**Europe** a la mortalité la plus haute (4,20 %) sur 56 pays. L'**Afrique** enregistre le taux de guérison le plus bas (57,01 %) sur 48 pays.

## *5. Analyse Prescriptive — Classification des Pays*



Analyse prescriptive - Recommandations par pays

Classification et recommandations - resultats reels Spark

Basee sur KPI 1 (mortalite) et KPI 2 (guerison) calcules par PySpark sur le cluster Hadoop

| PAYS | KPI 1 MORTALITE | KPI 2 GUERISON | CATEGORIE | RECOMMANDATION |
|---|---|---|---|---|
| Yemen | 28.56% | 49.26% | Intervention urgente | Deploiement immediat de ressources medicales |
| United Kingdom | 15.19% | 0.48% | Intervention urgente | Deploiement immediat de ressources medicales |
| Belgium | 14.79% | 26.27% | Intervention urgente | Deploiement immediat de ressources medicales |
| Italy | 14.26% | 80.64% | Intervention urgente | Deploiement immediat de ressources medicales |
| France | 13.71% | 36.86% | Intervention urgente | Deploiement immediat de ressources medicales |
| Hungary | 13.4% | 74.84% | Intervention urgente | Deploiement immediat de ressources medicales |
| Netherlands | 11.53% | 0.35% | Intervention urgente | Deploiement immediat de ressources medicales |
| Mexico | 11.13% | 76.82% | Intervention urgente | Deploiement immediat de ressources medicales |
| Spain | 10.44% | 55.2% | Intervention urgente | Deploiement immediat de ressources medicales |
| Canada | 7.68% | Non rapporte | Intervention urgente | Deploiement immediat de ressources medicales |
| Sweden | 7.18% | Non rapporte | Intervention urgente | Deploiement immediat de ressources medicales |
| Ecuador | 6.82% | 43.8% | Intervention urgente | Deploiement immediat de ressources medicales |
| Ireland | 6.81% | 90.24% | Intervention urgente | Deploiement immediat de ressources medicales |
| Sudan | 6.3% | 51.99% | Intervention urgente | Deploiement immediat de ressources medicales |
| Liberia | 6.17% | 55.36% | Intervention urgente | Deploiement immediat de ressources medicales |
| Niger | 6.1% | 90.72% | Intervention urgente | Deploiement immediat de ressources medicales |
| Switzerland | 5.74% | 89.62% | Intervention urgente | Deploiement immediat de ressources medicales |
| Slovenia | 5.56% | 83.04% | Intervention urgente | Deploiement immediat de ressources medicales |
| Iran | 5.42% | 86.9% | Intervention urgente | Deploiement immediat de ressources medicales |
| China | 5.37% | 90.88% | Intervention urgente | Deploiement immediat de ressources medicales |
| Egypt | 5.03% | 37.67% | Intervention urgente | Deploiement immediat de ressources medicales |
| Mali | 4.93% | 76.12% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |
| Burkina Faso | 4.82% | 84.18% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |
| Indonesia | 4.82% | 58.0% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |
| Romania | 4.81% | 56.19% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |
| Greece | 4.78% | 32.51% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |
| Peru | 4.73% | 69.93% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |
| North Macedonia | 4.56% | 54.48% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |
| Finland | 4.45% | 93.54% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |
| Denmark | 4.45% | 91.6% | Vigilance requise | Renforcer le systeme de soins et augmenter les tests |

La classification prescriptive, basée sur le croisement des deux KPIs, permet de catégoriser les 187 pays en quatre groupes d'action. Les **29 pays en intervention urgente** comme le Yémen, le Royaume-Uni ou la Belgique nécessitent un déploiement immédiat de ressources médicales. Les **54 pays en vigilance requise** doivent renforcer leurs soins et leurs tests. Les **55 pays en situation modérée** sont sous surveillance accrue. Enfin, les **49 pays en situation contrôlée** comme Djibouti ou l'Islande démontrent qu'une gestion rigoureuse .

# 8.Conclusion

Ce projet a permis de démontrer concrètement comment les technologies Big Data — Apache Spark 3.4.3 et Hadoop 3.3.5 déployés sur un cluster Docker — peuvent être appliquées à une problématique de santé publique réelle et mondiale. En partant d'un dataset brut de 187 pays issu de Kaggle et alimenté par les données de l'Université Johns Hopkins, nous avons construit de bout en bout une pipeline complète : infrastructure distribuée, ingestion dans HDFS, traitement PySpark, analyse multi-niveaux et visualisation interactive via un dashboard Flask.